

Pandas Cheat Sheet DSC 270 F24

Getting Started:

```
import pandas as pd
df.groupby?      # View docstring for groupby
help(df.groupby) # Full documentation
df.grou          # Use tab to autocomplete
```

Shell Commands in Jupyter:

You may need these to access data files or install packages.

```
pwd          Print Working Directory: display current path
cd           Change Directory to Jupyter default directory
cd lab4      Change to a subdirectory such as "lab4/"
ls           List files in the current directory
pip install seaborn  Install a new Python package
```

Types in Pandas:

DataFrame: 2D labeled, size-mutable data structure
Series: 1D labeled array
Index: Immutable labels for rows/columns
MultiIndex: Hierarchical index for higher-dimensional data

Creating a Pandas Series:

From a list (or iterable):

```
ser = pd.Series([10, 20, 30])
ser = pd.Series(range(1,8,2), dtype=float)
ser = pd.Series(np.linspace(0, np.pi, 180))
```

With custom index:

```
names = ['Alice', 'Bob', 'Claire']
values = [98., 87., 92.]
ser = pd.Series(scores, index=names)
```

From dictionary (or subset):

```
d = {'Alice': 98, 'Bob': 87, 'Claire': 79}
ser = pd.Series(d)
sub = pd.Series(d, index=['Alice', 'Bob'])
```

Inspecting a Series:

```
ser = df['A'] # Extracts series A from DataFrame df
ser.dtype    # Type of the elements: float64
ser.shape    # Shape as tuple: (100,)
ser.unique()  # Unique values for categorical series
ser.nunique() # Number of distinct values in series
ser.value_counts() # Count of each unique value
```

Accessing Data in a Pandas Series:

Note: An index returns an item; a slice or list returns a Series.

Label-based indexing and slicing with .loc[] and .at[]:

```
ser.loc['Alice']      # Access by label
ser.loc[['Alice', 'Bob']] # List of labels
ser.loc['Alice':'Bob'] # Slice of labels
ser.loc[:'Bob']        # Slice of labels
ser.at['Alice']        # Fast lookup (scalars only, no slices)
```

Position-based indexing with .iloc[]:

```
ser.iloc[0]           # Access by pos from 0
ser.iloc[[0, 2]]       # List of positions
ser.iloc[1:3]          # Slice (position 1 and 2; not 3)
ser.iloc[:3]           # Slice (positions before 3)
ser.iat[0]             # Fast lookup (scalars only, no slices)
```

Legacy indexing with square brackets (discouraged):

```
ser['Alice'] # gets item at index 'Alice' of Series
ser[0]       # deprecated unless index 0 item exists
```

Creating a Pandas DataFrame:

Import a CSV file on the local machine or on the web:

```
df = pd.read_csv("data_file.csv")
df.read_csv('https://example.com/data.csv')
```

Import a Microsoft Excel File on the local machine:

```
df = pd.read_excel('your_file.xlsx')
```

Create a DataFrame from a Python dictionary:

```
d = {'Alice': [98, 90, 100],
      'Bob':   [87, 85, 93],
      'Claire': [79, 68, 100]}
df = pd.DataFrame(d)
df = pd.DataFrame({'col1': [1, 2, 3],
                   'col2': ['a', 'b', 'c']})
```

Inspecting a DataFrame:

```
df.head(5) # Display only the first 5 rows
df.tail(5) # Display only the last 5 rows
df.index   # Index of DataFrame
df.columns # Columns of DataFrame
len(df)    # Number rows/instances: 100
df.shape   # Shape as tuple: (100, 5)
df.ndim    # Number of dimensions: 2
df.size    # Total number of elements: 500
df.info()  # Counts of rows, cols, and values
df.describe() # Statistical summary of data
df.sample(10) # Gets random sample of 10 rows
```

Type Conversion and Copying:

```
.copy()      Create deep copy of a DF or Series
df_copy = df.copy()

.astype()    Convert Series or DF data to specified type
df['Column'] = df['Column'].astype(float)

.to_numpy()  Convert DF or Series to a NumPy array
arr = df.to_numpy()

.item()      Get single value from scalar-like Series or DF
value = df['Column'].iloc[0].item()
```

Reshaping DataFrames:

```
.melt(): Convert wide-format DF to long format
df_melted = df.melt(id_vars='id_vars', value_vars='value_vars')

.pivot(): Reshape data to wide format (inverse of melt)
df_pivoted = df.pivot(index='index_col', columns='columns_col',
                       values='values_col')

.concat(): Concatenate DataFrames along rows or columns
df_combined = pd.concat([df1, df2], axis=0)

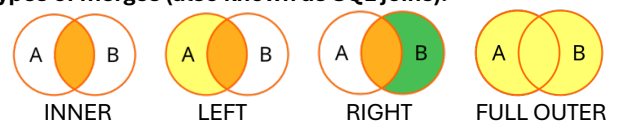
.transpose(): Swap rows and columns
df_transposed = df.transpose() # or df.T

.stack(): Pivot columns into rows (creates MultiIndex).
df_stacked = df.stack()

.unstack(): Pivot rows into columns (opposite of stack).
df_unstacked = df.unstack()

.merge(): Combine DFs based on keys (like SQL JOIN).
Note: If non-key cols in A and B have same name, use suffixes=.
df_merged = pd.merge(df_left, df_right,
                     on='key', how='inner')
```

Types of merges (also known as SQL joins):



Accessing Data in a DataFrame

Depending on the operation, you may get a DF, Series, or scalar value.

Column Selection:

By column name:

```
df['column_name'] # Access single column (as Series)
df[['c1', 'c2']] # Access multiple columns (as DF)
```

Attribute access (if column name is a valid identifier):

```
df.column_name # Equivalent to df['column_name']
```

Label-based indexing and slicing with .loc[], .at[]:

Access rows by label:

```
df.loc['row_label'] # Access single row (as Series)
df.loc[['row1', 'row3']] # List of row labels (as DF)
df.loc['row1':'row3'] # Slice of rows (inclusive)
df.loc[:'row3'] # Slice from start thru 'row3'
```

Access specific rows and columns:

```
df.loc['row1', 'col1'] # Scalar value at row and col
df.loc[['row1', 'row2'], ['col1', 'col2']] # Subset of rows/cols
df.loc['row1':'row3', 'col1':'col3'] # Slice rows and columns
```

Access all rows for specific columns:

```
df.loc[:, 'col1'] # All rows, single column (as Series)
df.loc[:, ['col1', 'col2']] # All rows, multiple columns
```

Fast scalar access:

```
df.at['row1', 'col1'] # Faster access to a scalar value
```

Position-based indexing with .iloc[] and .iat[]:

Access rows by position:

```
df.iloc[0] # Access first row (as Series)
df.iloc[[0, 2]] # List of row positions
df.iloc[1:3] # Slice of rows (positions 1 and 2)
df.iloc[:3] # Slice from start to position 2
```

Access specific rows and columns:

```
df.iloc[0, 0] # Scalar value at first row and col
df.iloc[[0, 2], [1, 3]] # Subset of rows/cols by position
df.iloc[0:3, 0:2] # Slice rows and columns
```

Access all rows for specific columns:

```
df.iloc[:, 0] # All rows, first column (as Series)
df.iloc[:, [0, 2]] # All rows, cols at positions 0 and 2
```

Fast scalar access:

```
df.iat[0, 0] # Faster access to a scalar value
```

Boolean masking (condition-based selection):

Note: See Filtering section for more detail.

Filter rows based on column condition:

```
df[df['col1'] > 90] # Rows where 'col1' > 90
df.loc[df['col1'] > 90] # Same as above, explicit with .loc[]
```

Legacy indexing and slicing with square brackets:

Note: While you can use [] for convenience, it is recommended to use .loc[] and .iloc[] to avoid ambiguity and potential issues.

Select rows by slice (position-based):

```
df[0:3] # Rows at positions 0 to 2
```

Select columns:

```
df['col_name'] # Access single column (as Series)
df[['col1', 'col2']] # Access multiple columns (as DF)
```

Filtering Data in Pandas DataFrame

Boolean Masking:

Basic Syntax:

```
df[condition]
```

Combine Conditions: Use & for **and**, | for **or**, ~ for **not**
MUST use with parentheses ()

Examples:

Single condition: df[df['Column'] > value]

Multi conditions: df[(df['Col1'] > val1) & (df['Col2'] < val2)]

Using Boolean masking with .loc[]:

Filters rows based on **Boolean mask** (must be a Series).

Optionally, columns can also be specified.

Syntax: df.loc[condition] df.loc[condition, columns]
df.loc[condition, ['Column1', 'Column2']]

Example: df.loc[df['Column'] > value]

Using .query() :

Filters rows based on a **SQL-like** condition: .query() is more readable for complex filtering compared to Boolean masks.

Syntax: df.query('expression')

Features:

Reference columns directly by name

Logical operators: and or not

Comparison operators: > < >= <= == !=

Variables: Use @ to include Python variables

Also supports **Column arithmetic** and function calls

String matching (e.g., df.query('Name == "Maria"')).

Examples:

```
df.query('Column > value and OtherColumn != "string"')
```

```
df.query('Column1 > Column2 + 10')
```

```
df.query('Column == @variable')
```

Remember to enclose string values in quotes in .query()!

Index and Sorting Operations in Pandas

Setting Index: .set_index(): Set column(s) as index

```
df = df.set_index('ColumnName')
```

```
df = df.set_index(['Col1', 'Col2']) # MultiIndex
```

Resetting Index: .reset_index(): Reset to default integer index

```
df = df.reset_index(drop=True) # Discard old index
```

Reindexing: .reindex(): Align to new index.

```
df = df.reindex(new_index)
```

Sorting by values with .sort_values():

```
df = df.sort_values(by='ColumnName')
```

Sorting by index with .sort_index():

```
df = df.sort_index()
```

Renaming Columns and Index Labels:

```
df = df.rename(columns={'old_col': 'new_col'},
               index={'old_idx': 'new_idx'})
```

Access Index Object as property with .index:

```
idx = df.index
```

Set Index Name(s):

```
df.index.name = 'IndexName'
```

```
df.index.names = ['Level1', 'Level2'] # For MultiIndex
```

MultiIndex: Creating from Columns:

```
df = df.set_index(['Col1', 'Col2'])
```

Resetting Levels in a MultiIndex:

```
df = df.reset_index(level='Col1')
```

Additional Tips:

```
df.loc['IndexLabel'] # Select row by index label
```

```
df.loc[['Label1', 'Label2']] # Select multiple rows
```

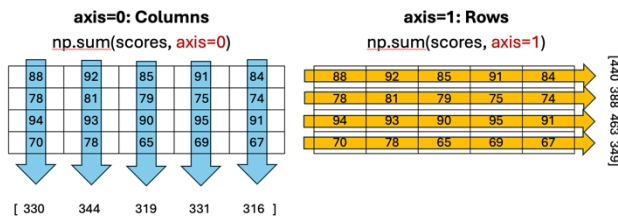
```
df.loc['start':'end'] # Slice rows between 'start' and 'end'
```

```
df.index.is_unique # Returns True if index has unique values
```

Aggregate Functions

```
df['A'].sum() # sum of all entries in column A
df['A'].prod() # product of all entries in column A
df.mean(axis=0) # means of all columns of df
df.min(axis=0) # smallest value of each column of df
df.max(axis=0) # largest value of each column of df
df.idxmin() # index of smallest value in each series/col
df.idxmax() # index of largest value in each series/col
df.var() # variance of each series
df.std() # standard deviation of each series
df.median() # middle value of each series if sorted
df.quantiles([.25, .75]) # quantiles of each series
df.count() # counts non-null values in each series
```

The Axes



```
df.mean() # means of columns/variables (the default axis)
df.mean(axis=0) # means of columns/variables
df.mean(axis=1) # means of the rows/instances
df.mean(axis=None) # mean of all entries in DataFrame
df.sum.sum() # sums all entries (axis=None doesn't work)
df.to_numpy().sum() # or you can always resort to this!
```

Note: By default, Pandas aggregates along columns (axis=0). Some functions, e.g., .sum(), do not permit use of axis=None.

Mathematical Operations in Pandas

Element-wise Arithmetic Operations:

Supported operators: +, -, *, /, ** (exponentiation), % (mod)

Operations are **broadcasted** over DataFrames and Series.

```
df1 + df2 # Add DFs element-wise
```

```
df['Column'] * 2 # Multiply col by scalar
```

Comparison Operators:

Operators: >, <, >=, <=, ==, !=

```
df['A'] > df['B'] # Element-wise comparison
```

```
df['C'] != 0
```

Boolean Operations:

Logical operators: & (and), | (or), ~ (not)

Use parentheses due to operator precedence

```
(df['A'] > 0) & (df['B'] < 5)
```

```
~df['C'].isna()
```

Element-wise Functions:

Most np functions are supported; operate element-wise

Examples: np.abs(), np.sqrt(), np.exp(), np.log()

```
df['A'].abs() # Absolute value
```

```
np.sqrt(df['B']) # Square root
```

Logical Reduction: .any(), .all(): evaluate condition along axis

```
df['A'].gt(0).any() # Check if any values > 0
```

```
df['B'].lt(5).all() # Check if all values < 5
```

Clipping to Force Values into a Valid Range:

```
df['A'].clip(lo, hi) # Limit values between lo and hi
```

```
df['A'].clip(lower=0) # Replace negative values with 0
```

Note: Operations between DFs and Series align on **labels** (index and columns). If alignment is not desired, consider using .to_numpy() to perform operations without alignment.

Grouping Data

Create a GroupBy object:

Group rows on the values of a single column:

```
df.groupby(by='grouping_column')
```

Group on all combinations of values in multiple columns:

```
df.groupby(by=['col1', 'col2', 'col3'])
```

Select the column or columns to aggregate on:

Note that you can select columns in any of the usual ways: single column name, slice, list, etc.

```
df.groupby(by='col1')['col3']
```

```
df.groupby(by=['col1', 'col2'])['col3']
```

```
df.groupby('col1')[['col3', 'col4']]
```

```
df.groupby(['col1', 'col2'])[['A': 'E']]
```

Apply an aggregate function to selected columns:

Use a single function to compute statistics for each group:

```
df.groupby(by='col1')['col2'].mean()
```

```
df.groupby('A', 'B')[['C', 'D']].min()
```

Use agg() for more complex aggregation tasks:

The agg() method takes a dictionary as its argument:

Keys: string names of individual columns

Values: string names of function or list of functions

```
df.groupby('col').agg({'col2': 'mean'})
```

```
df.groupby('A', 'B').agg({
```

```
    'C': 'min',
```

```
    'D': ['mean', custom_function]
```

```
})
```

Custom Aggregation Functions

- Takes pd.Series as argument, returns scalar

- Uses vectorized operations as fully as possible

- Handles missing values in an appropriate manner

```
def my_range(series):
    return series.max() - series.min()
```

```
grouped = df.groupby('group_col')
result = grouped.agg({'num_col': my_range})
```

Can also pass an anonymous instead of named function:

```
(df.groupby('category')
 .agg({'col': lambda x: x.max() - x.min()}))
```

Applying Functions with .apply()

DF .apply(): Apply function along an axis (rows or columns)

```
df.apply(func, axis=0) # Apply to each column (default)
```

```
df.apply(func, axis=1) # Apply to each row
```

Series .apply(): Apply a function element-wise.

```
df['Column'].apply(func)
```

Parameters:

func: Function to apply (can be built-in, NumPy, custom)

axis: Axis along which function is applied (0=cols, 1=rows)

Examples:

Aggregate Functions:

```
df.apply(np.sum) # Sum of each column
```

```
df.apply('mean', axis=1) # Mean of each row
```

Custom Function on Rows:

```
df.apply(lambda row: row['A'] + row['B'], axis=1)
```

Apply Function to Series:

```
df['Col'].apply(np.sqrt) # Square root of each element
```

```
df['Col'].apply(lambda x: x**2)
```

Notes: 1. When using .apply() on DF with axis=1, the function receives each **row** (as a Series). 2. For element-wise operations on a DataFrame, consider using .applymap(). 3. Use vectorized ops or built-in functions when possible for better performance.

Handling Missing and Duplicated Data

*Note: These methods can help you clean and preprocess your data before analysis. ⚠️ **CAUTION:** Think carefully about **why** the data is missing and the statistical impacts of your approach!*

Detecting Missing Data:

.isna() / .isnull(): Returns bool DF/Series (True if missing)
df.isna() df['Column'].isnull()
.notna() / .notnull(): Opposite of .isna(); True if NOT missing
df.notna() df['Column'].notnull()

Drop Missing Data:

.dropna(axis=0): Drop rows with NaNs (axis=1 for columns)
df.dropna()
df.dropna(subset=['Col1', 'Col2'])

Fill Missing Data:

.fillna(value): Replace NaNs with specified value
df.fillna(0)
df.fillna(method='ffill') # Forward fill

Detecting Duplicated Data:

.duplicated(): Boolean Series of duplicate
df.duplicated()
df.duplicated(subset=['Col1'])

Drop Duplicates:

.drop_duplicates(): Remove duplicate rows.
df.drop_duplicates()
df.drop_duplicates(subset=['Col1'], keep='last')

Conditional Replacement:

.where(condition, other): Replace where condition is **False**
df.where(df > 0) # Keep positives, NaN elsewhere
df.where(df > 0, other=0) # Replace negatives with 0
.mask(condition, other): Replace where condition is **True**
df.mask(df < 0, other=0) # Replace negatives with 0

Additional Methods:

.replace(): Replace specified values.
df_replaced = df.replace(to_replace=np.nan, value=0)
.interpolate(): Fill NaNs using interpolation.
df_interpolated = df.interpolate(method='linear')

Time Series

Converting to Datetime:

df['Date'] = pd.to_datetime(df['Date'])

Setting Datetime Index:

df = df.set_index('Date')

Date Range Generation:

dates = pd.date_range(start='2020-01-01', periods=100, freq='D')

Resampling Time Series:

df_resampled = df.resample('M').mean() # Resample to monthly freq
df_resampled = df.resample('W').sum() # Resample to weekly freq

Time-based Indexing and Slicing:

df['2020-01-01':'2020-03-31'] # Slice by date range
df.loc['2020'] # All data from the year 2020

Shifting and Lagging Data:

df.shift(1) # Shift data forward by one period
df['Diff'] = df['Value'] - df['Value'].shift(1) # Compute difference

Rolling Window Calculations:

df['RollingMean'] = df['Value'].rolling(window=7).mean()

Timezone Handling:

df = df.tz_localize('UTC') # Localize to UTC
df = df.tz_convert('US/Eastern') # Convert to another timezone

Handling Missing Dates:

df = df.asfreq('D') # Set freq to daily, introducing NaNs for missing dates
df = df.fillna(method='ffill') # Forward fill missing values

Remember: Ensure date column is in datetime format and set as the index
Common Freqs for Resampling: Day(D) Week(W) Month(M) Qtr(Q) Year(A)

Modifying DFs: Inserting, Deleting, Assignment

Inserting Columns:

Via assignment:

df['Volume'] = df.Length * df.Height * df.Depth

With .assign() (allows chaining):

df = df.assign(Area=lambda df: df.Length * df.Height)

At specific position with .insert():

df.insert(loc=1, column='new_column', value=values)

Inserting Rows:

Via assignment with .loc[]:

df.loc[new_index_label] = values

Using pd.concat():

df = pd.concat([df, new_row_df], ignore_index=True)

Deleting Columns and Rows:

Delete column:

df.drop(columns='col_name', inplace=True)

Delete row:

df.drop(index=row_label, inplace=True)

Assignment and Transformation:

Modify data:

df.loc[row_label, 'col_name'] = new_value

Binning with pd.qcut():

df['binned'] = pd.qcut(df['col'], n, labels=False)

Additional Notes:

Avoiding the Dreaded SettingWithCopy Warning:

Use .loc[] to ensure you're modifying the original DF.
df.loc[df['A'] > 0, 'B'] = new_value

Creating a Copy:

Use .copy() to mod a copy without affecting the original
df_copy = df.copy()

Index and MultiIndex

Create from Column: df = df.set_index('Column')

Resetting Index to Default: df = df.reset_index()

Creating a MultiIndex: df = df.set_index(['Col1', 'Col2'])

Creating MultiIndex from Tuples: tuples = [('A', 1), ('A', 2), ('B', 1)]
df.index = pd.MultiIndex.from_tuples(tuples, names=('Group', 'Number'))

String Operations in Pandas

Access string methods using .str:

df['col'].str.method()

Common String Methods (apply to Series as shown above):

Lowercase: .str.lower()
Uppercase: .str.upper()
Strip whitespace: .str.strip()
Contains substring: .str.contains('sub')
Replace substring: .str.replace('old', 'new')
String length: .str.len()
Split strings: .str.split('sep')
Extract with regex: .str.extract('pattern')

df['name'].str.upper() # Convert to uppercase
df['email'].str.contains('@domain.com') # Check for domain
df['phone'].str.replace('-', '') # Remove hyphens
df['words'].str.split(' ') # Split on spaces

Note: These return Series; can assign result back to a column if needed.

Plotting with Pandas

Histogram: df['total_bill'].plot.hist()

Scatter Plot: df.plot.scatter(x='total_bill', y='tip')

Line Plot/Time Series: df['Value'].plot()